

Document intelligence

How a file becomes a record, and when it must stop
before it does.

Andre Naidoo

Technical assessment · September 2026

Desk on this machine · 127.0.0.1:8787

Confidential

THE JOB

A wrong total, written as final, is the failure

A run that stops and says why is a success. That is the policy. The rest of this document is how I keep it true when nobody is watching, and how I prove it on the desk.

The file can be a contract, an invoice, a brief, or a signed agreement. Someone drops it. The system has to say what it is, who it is with, what it is worth, and when it applies. If it cannot say that from the page, it holds the file. It does not fill the gap.

Five rules sit under every step.

1. A required value is kept only when it is on a page I can point at.
2. Every failure ends in a state a person can see. There is no path that continues with a hole.
3. The file can sit unattended. Received is not approved.
4. The thread and the row say what happened, and neither can be quietly overwritten.
5. Who the person is, and which rows they may see, are two different checks.

A document is done when the file is stored, the identity is settled, the read has finished, the required fields have passed, each of those fields has a page, the duplicate and revision checks have passed, the current-row write has committed, the downstream view has been updated or queued, and an audit line has been written. A green execution in n8n is not that list. n8n decides the next step. It does not own the rows or the files.

The desk proves the state changes that matter: a clean invoice, the same file twice, a revised total, and a blank page. Production is the same machine with a malware scan, a real login, OCR, page chunks, and doors in Slack, Drive, and a mailbox. I have not built those doors, and I do not describe them as running.

THE LIFE OF A ROW

Received, then a decision

The database row starts as received. The worker claims it as processing. From there it becomes accepted, held, or waiting. Accepted with nothing pointing at a newer row is current. Waiting is a yes or a no. Silence stays waiting.

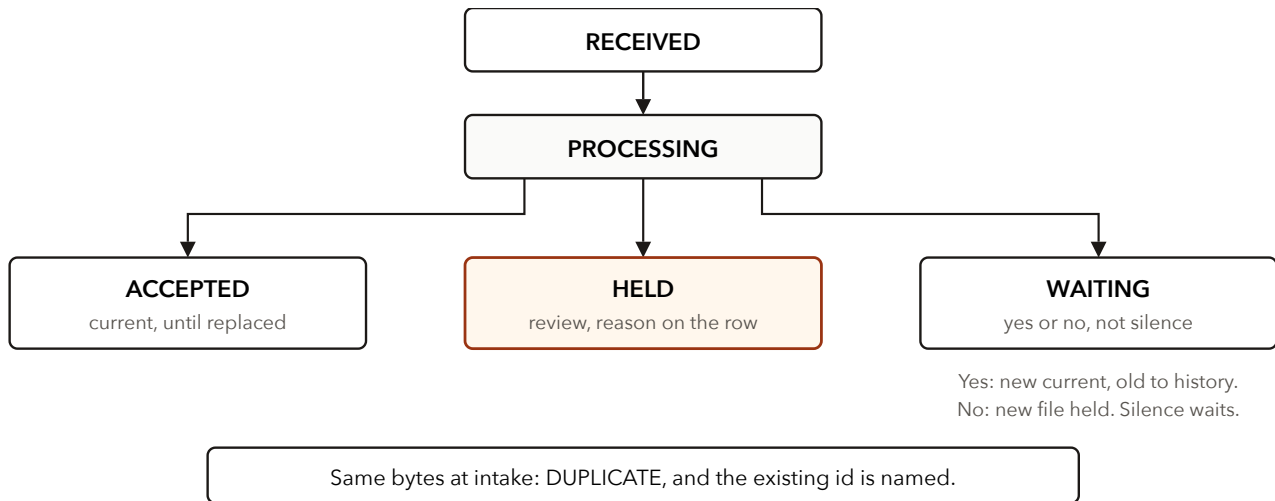


Figure 1. One row, one current record. A duplicate never becomes a second document.

The same bytes are recognised before a model call. The hash is unique in Postgres. If two uploads of those bytes arrive together, one insert wins and the other is told which document it already is. A different file with the same invoice number or contract reference does not overwrite. It waits. Yes replaces the current row and moves the old one to history in one transaction. No holds the new file. If that transaction fails, nothing has moved.

WHERE THE WORK SITS

The desk, the queue, the book

A person opens a page on this machine. They do not open n8n. The page knows who the link is for, asks what the file is, and shows the book: reading, current, held, history. Four cases sit on that page. Each one runs the real intake.

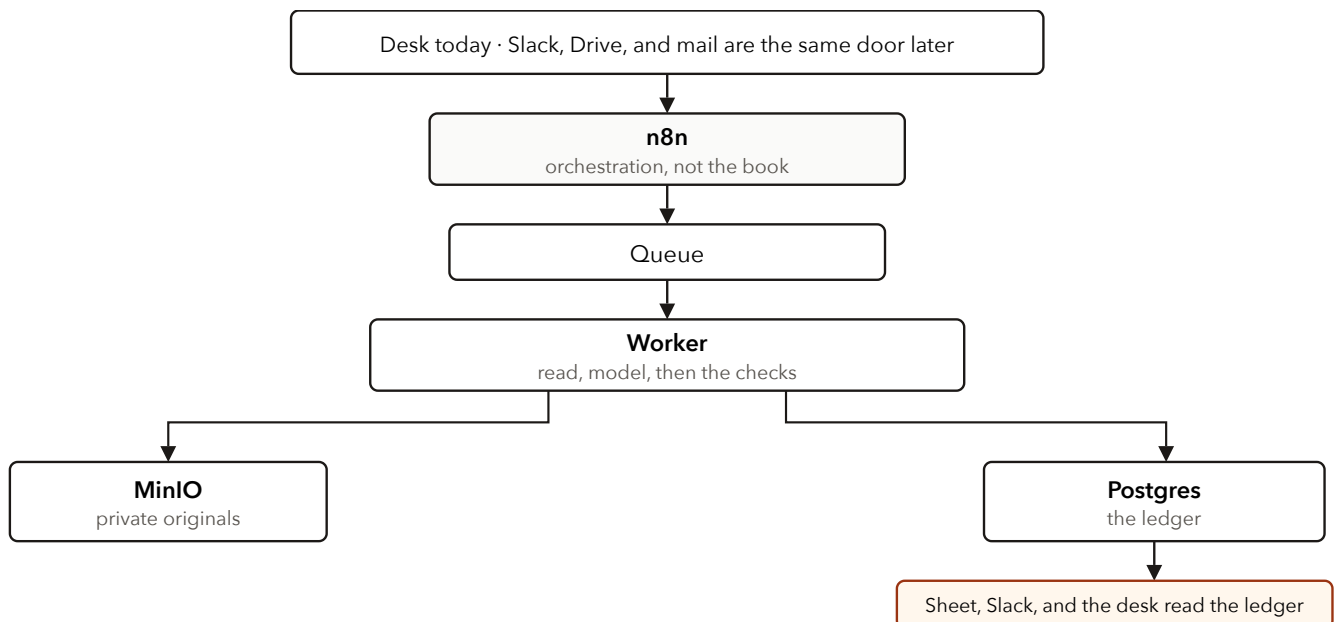


Figure 2. n8n starts the work. The file lives in the bucket. The row lives in Postgres.

On this machine the path is already that shape. Intake hashes the file. The same hash is named and stopped, with no model call. A missing type, a missing department, or a file that is not a PDF, PNG, or JPEG is asked about and stopped. Otherwise the original goes to a private bucket, a received row is

inserted, the job is queued, and the person is told the file is not approved yet. The worker then claims the row, reads it, and writes accepted, held, or waiting. The browser is not holding the HTTP call open for the model.

The order I want in production, once the scan is in front of the bucket, is: upload, malware scan, durable storage, hash, one database insert, then the queue. The scan is not in this build. ClamAV on the same machine is the first version I would add. A flagged file stays received and is not opened.

ON THIS BUILD

You can run it

- Desk, thread, and the four cases
- Hash, and a unique row in Postgres
- Private bucket for the original
- Queue, worker, received then processing
- Accepted, held, waiting, history
- Page check on required fields
- Yes or no on a whole revised row
- Department filter on the book

WRITTEN, NOT RUNNING

Same rules, more doors

- Google login, instead of the link
- Slack, Drive, and a known mailbox
- Malware scan before the file is opened
- OCR only where the page has no text
- Chunks, with the page kept on each field
- A correction of one field, not the whole row
- A Google Sheet of current rows only
- Alerts when the queue or the worker stalls

PART A

Foundations

A1. Webhook, and a manual or form trigger

A webhook is a URL another system calls. The desk, and later Slack or a mailbox, already has the file. The workflow starts because that system posted it, or posted a pointer to it.

A manual trigger is the button inside n8n. I use it while I am building. It is not how finance submits a contract. A form trigger is a page n8n hosts. I would use it if I had no other page. I built the desk because the thread, the book, and the yes or no need to live on one screen. I do not want the person on an n8n URL.

A2. Waiting, and not waiting

Synchronous means the caller sits until the work is finished. Asynchronous means the caller is told the file was taken, and the work happens after that call has ended.

Intake on this build is the second kind. It stores the file, inserts received, queues the worker, and replies. The reply is that the file is not approved yet. The case buttons on the desk then watch the book until the row leaves reading. A 100-page file takes the same first reply. It must not hold a browser open. Chunking that file is still to build. The reply is not.

A3. A 429 from the model

HTTP 429 means the service asked us to slow down. It is not a bad file. A 400 is a bad request. A 401 is a bad key. I do not retry those. Retrying them makes noise and can hide a broken configuration.

The model call on this build tries four times, three seconds apart. That is a start. The policy I want is: honour Retry-After when the service sends it, then wait longer each time with a little randomness, and stop at a retry budget. Past that budget the row is held and the thread says the model service asked us to wait. Fields stay blank. An empty total is not zero.

A retry is the same document id. It must not insert a second row, and it must not accept a second current record. The id is assigned at intake, before the model is called.

A4. Self-hosted n8n, against n8n Cloud

The contracts stay on a machine we run. Queue mode, Postgres, and the bucket sit next to n8n. Execution data can contain page text. I do not want that text on someone else's cloud as the price of not running a box. We are also not billed per execution while a worker reads a long file.

The cost of that choice is the box. Upgrades, backups, disk, the encryption key, and a worker that has quietly stopped are ours. Cloud would take that class of failure off the table. For this workload I still self-host, and I treat backups of Postgres and the bucket as part of the system.

A5. Idempotent

Doing the thing twice leaves the same result as doing it once. A double click, a retry after a timeout, a mailbox and a person both submitting the same PDF: one document, not two.

The key is the hash of the bytes. The column is unique. The lookup happens before the model. If two inserts race, one wins and the other receives the existing id. A worker that dies after the row is already accepted does not write it again. The update runs only while the row is received or processing. A second delivery of a finished document is skipped.

WHO MAY SEE IT

Security and access

Authentication says who the person is. Authorization says which documents and which fields they may see. On this desk, the link `?as=dre` stands in for the first of those. It is a named seat in a staff map, so the department filter and the thread can be shown. It is not a login. Production removes it. The person authenticates with Google, or with the Slack identity already stored against that person, and the book is checked before the original file or a sensitive field is returned.

The header secret on the webhook stops a stranger calling n8n directly. It does not prove who the person is. The browser never receives the database password, the model key, or the storage key. Those sit outside the workflow data and can be rotated on their own.

Legal's query does not return payment amounts. The page does not merely hide a column. A superuser sees the set. Nobody gets a delete. A wrong row is superseded and stays in history. The thread is append-only. Documents and rows are kept for a retention period I would write down with the company before go-live. I have not invented that period here.

WHAT MAY BECOME A RECORD

The model proposes. The checks decide.

I do not treat the model’s reply as the record. It returns one JSON object. Temperature is zero. The instruction says to leave a field empty when it is not on the page, and to list a clash instead of picking a winner. Ordinary code then decides whether those values are allowed to become truth.

TASK	WHO DOES IT
File hash	Code
Duplicate detection	Postgres, unique hash
Who the person is	Identity provider. On this desk, the link is a stand-in.
Which rows they may see	Code and the book
OCR	OCR engine, on pages with no text. Not in this build.
Reading the fields	The model
Naming a conflict	The model lists it. Code refuses to pick.
Date, amount, required fields	Code
Replacing the current row	One Postgres transaction
Retry	The workflow, inside the same document id
The queue	Redis and the worker
A yes or a no	The person
The audit line	Postgres

Gemini 2.5 Flash, through OpenRouter, is the model this build calls. It takes text and a single image, which is why a photo of an invoice can already be sent. On an invoice the counterparty is the supplier, not the customer. If the person marked the type and the text looks like a different type, I hold the file and I say so. I do not relabel it. The type decides which fields are required.

For a text PDF, a required value has to appear on a page. If it does not, the field is blanked and the document is held. The row records the sentence: the total, the reference, the date, the party, and the currency, each with its page. A total with no page is not a total. A picture skips that text check, because there are no page strings to search. That is a gap I would close with OCR, not by trusting the model more.

The system does not assume the model output is correct. Unsupported or conflicting values are rejected. The model output is a candidate extraction, not the book.

THE ROW TEAMS READ

One document, one current line

Postgres is the book. A Google Sheet, when I connect it, is a view of rows that are current and accepted. Held rows do not appear on the finance sheet. History is a second tab, so a revised total does not look like two live invoices. If the sheet is down, the book is still the book, and the sync retries. The sheet is not where the workflow writes first. It is not connected on this build.

COLUMN	WHAT IT IS
document_id	Stable id, from the hash and a time suffix. Assigned at intake.
type, department	The type they stated. The department comes from the staff map.
parties, counterparty	Parties are required on a contract. On an invoice, the counterparty is the supplier.
dates	Document, effective, expiry. A real calendar day, or blank. Effective date is required on a contract.
currency, total	Digits and a decimal. No symbol in the amount. Both required on an invoice.
reference	Invoice number or contract reference. This is how a revision is recognised.
why it exists	Required on a brief.
submitted_by, accepted_at	Who sent it, and when it became current.
original	A storage key. The desk streams it only after the row says this person may see it.

The ledger also keeps the hash, the storage key, page notes, validation notes, which model ran, who reviewed it, and the id of the row this one replaces. An invoice needs the supplier, the invoice number, the currency, the total, and the invoice date. A contract needs the parties and the effective date. A brief needs the title and why it exists. Anything marked other needs a title.

WHEN IT STOPS

Every failure has a visible end

I group failures so a reviewer can see there is no silent branch.

CLASS	EXAMPLES	WHERE IT ENDS
Input	Unsupported file, corrupt file, malware, missing type or department	Asked, or refused, before a record is approved. A scan failure stays received and unopened.
Extraction	OCR failure, model timeout, junk JSON, provider down	Held. Fields blank. The thread says the read did not return a usable result.
Validation	Missing field, bad date, bad amount, two values	Held. The bad field stays blank. Both conflicting values stay in the note. Nothing is picked.
Business rule	Duplicate, revision, more than one current match	Duplicate names the old id. A revision waits. An ambiguous match is held and the ids are listed.
Infrastructure	Database down, bucket down, worker down, queue growing	The job stays queued or fails in the open. It does not become current.

Every class has a visible terminal state. There is no path where the workflow continues with incomplete data.

ERROR	RETRY	THEN
429	Yes. Retry-After, then a longer wait, inside a budget.	Hold, and say the service asked us to wait.
408, timeout, 5xx	Yes, limited.	Hold if the budget is spent.
400	No	Hold, and inspect the request.
401, 403	No	Alert. This is a key or a permission, not a bad PDF.
Junk JSON	Once or twice, then stop.	Hold. Never accept.
OCR empty	Limited, then one picture of that page.	Hold and name the page if it is still empty.
Database unavailable	Yes	Leave the job on the queue.

This build retries the model call four times, three seconds apart. The longer wait, the randomness, and the Retry-After header are the policy above. They are not in the node yet. What is in the node is the rule that a failed call does not become an accepted row.

If the worker dies halfway through a document, the row is still processing. The queue’s visibility window expires and the same document id is delivered again. The write runs only while the row is received or processing. A crash cannot create a second current record. A crash after the row is already accepted is a skip, not a second approval.

LONG FILES, AND THE BILL

A contract is a set of pages

A one-page invoice is one text block, one model call, one decision. A 100-page contract is a pile of pages. Some have text, some are scans, some are blank on purpose. The person already has the reply that the file is received and not approved. The job id is the document id. Sending the file again hits the same hash.

The worker I would run walks the pages. Text stays text. A page with no text goes to OCR. If OCR is still empty, one picture of that page, then a hold that names the page. I do not send a hundred-page scan to the vision model page after page. Pages are then grouped, about ten at a time. Each group is its own call, and each candidate keeps its provenance: which chunk, which pages, which field, which value, which source page, and a short excerpt. The merge step does not see “chunk 4 says 18,000”. It sees “chunk 4 says 18,000, from page 37”. If two chunks disagree, both values are kept and the file is held.

That chunking is not in this build. The blank-page rule is. A contract whose second page has no text is held, and the note names page 2. I would rather hold a good invoice that sat next to a blank exhibit than accept a total from a file I could not see all of.

Cost is kept down by doing the cheap work first. The hash and the duplicate check are code. OCR runs only where the page has no text. The model runs only where a person would otherwise have to interpret the page. The price list is secondary, and it moves. Prices checked on 22 September 2026. Flash is 0.30 US dollars per million input tokens and 2.50 per million output tokens. Thinking tokens are billed as output. Flash-Lite is 0.10 and 0.40.

JOB	ROUGH SIZE	ROUGH COST, THINKING OFF
One-page invoice	~600 in, ~400 out	under 0.2 US cent
100-page contract, one call	~60,000 in, ~800 out	about 2 US cents
Same contract, ten chunks	ten smaller calls	still under 10 US cents
10,000 text documents, ~8 pages	~48 million in	on the order of 30 US dollars

Those counts are estimates from a typical page, not a measured bill from this project.

I split a long contract so the merge can refuse a clash, and so the browser is not the thing waiting. A few cents is not the reason. Thinking left on can also turn a cheap call into a much larger output bill, for a job that is a form. I would turn thinking off for the field pass, and I would measure that against the fixtures before I trusted it. Flash-Lite is the trial model for the cheap questions: is this page empty, does the stated type look wrong. I would not move the money fields to Lite until the same fixtures come back with the same holds and the same totals.

PART C

A person, a backfill, a quiet morning

A correction without opening n8n

A waiting row asks yes or no on the desk. That is enough for “does this revised invoice replace the one on file?” It is not enough when one field on page 12 is wrong and the rest of the contract is fine. For that, the held row should open as a short list: the value, the page, the excerpt, and a box. Approve runs the same date and money checks again. It does not call the model again. The row becomes current only when the required fields pass. That field-by-field screen is not on this build. The yes or no on the whole row is.

Ten thousand old files

I would not push an archive through the afternoon webhook. Intake is the same function: store, hash, insert received, or recognise a duplicate. The queue is a second, slower queue. Daily work stays on a fast queue with a small concurrency. The backfill yields when the fast queue has anything waiting. A file that is already current hits the unique hash and is skipped, so the archive can overlap a live day. I would pause the backfill on a run of 429s, or if the oldest daily job grows past a few minutes, and I would release it a few hundred files at a time after looking at the hold rate.

Healthy, without the execution log

The desk already shows whether n8n, the book, Redis, and the bucket answer. That is a port check, not a deep probe. The board I want adds six counts: jobs waiting, the age of the oldest job, holds and how long the oldest hold has sat, failures today, model spend today, and pages OCR could not read. Accepted rows are a count, so you can see that work is finishing. The sheet is where finance looks. The board is where I look to see whether the sheet is being lied to. Those alerts are not posting yet. The status line on the desk is the piece that is.

PROOF

Production acceptance

The primary safety measure is the false acceptance rate. A document counts as a false acceptance if it is marked current and a required value is wrong, or is not supported by the page. A false hold is a good file stopped for a person. A false acceptance is the more serious of the two. The system is biased toward holding an uncertain document.

I would not use the model's own sense of certainty as that proof. Each fixture has an expected value for every required field, including the page. Model output is compared with that expected result before a model, a prompt, or a version is promoted. A change that improves one field and degrades another is not promoted on its own. The measures I would keep are field accuracy, required-field recall, false acceptance rate, false hold rate, OCR failure rate, duplicate rate, processing time, and cost per document. I do not have a scored set large enough to publish a rate from, and I will not invent one.

TEST	EXPECTED RESULT	THIS BUILD
Valid invoice	Accepted, and current	Shown. INV-1001, 12500.00 ZAR, page 1.
Same file twice	One document, one model path	Shown. The existing id is named.
Two identical uploads at once	One insert wins. The other names it.	The unique hash is in Postgres. Not a button.
Revised invoice	Waiting. The current row is unchanged.	Shown. 18000.00 waits on 12500.00.
Revision approved	New row current. Old row to history, in one transaction.	The yes button calls that transaction.
Revision rejected	New file held. Previous row stays current.	The no button. Silence does neither.
Missing required field	Held. The field stays blank.	In the checks. No separate fixture button.
Conflicting totals or dates	Held. Both values kept.	In the checks. Nothing is picked.
Invalid date or amount	Held.	In the checks.
Blank or unreadable page	Held, and the page is named.	Shown. Page 2 of the contract fixture.
Malformed model JSON	Held. Never accepted.	In the checks, when the reply is junk.
Model timeout or 429	Retry inside the policy, then hold.	Four tries, three seconds apart.
Worker unavailable	The queue grows, and an alert fires.	The queue is real. The alert is not.
100-page document	Asynchronous. No browser timeout.	The reply is already asynchronous. Chunks are not.
Unauthorized person	No document, and no restricted field.	Department filter. The link is not a login.
Sheet unavailable	The book remains Postgres. Sync retries.	The sheet is not connected.

The system is not production-ready until these cases pass against the same build that is demonstrated.

The regression set

Functional: a clean invoice, a contract, a brief. Adversarial: a blank page, a corrupt PDF, conflicting totals, an impossible date, junk JSON. Concurrency: two identical uploads at once, two revisions at once, a worker restart during processing. Dependencies: a 429, a timeout, a dead model, a dead

database, a dead bucket. Scale: one 100-page document, a 10,000-file backfill, and live documents at the same time. Security: a person outside the department, a session that has expired. The four buttons cover the first slice of that list. They are the cases I will not weaken to make a demo look finished.

WHAT YOU CAN PRESS

The desk, and what it is not

The page is on this machine, at 127.0.0.1:8787. It is a test desk, not a public site. The status line shows whether n8n, the book, Redis, and the bucket answer. The buttons are Clean invoice, Same file again, Revised invoice, and Blank page. After a run the thread says whether the row matched the fixture.

A clean invoice comes back accepted, with the supplier, the invoice number, the total, and the page each value was found on. The same file comes back as the same document. A revised total comes back waiting, with the old total and the new one both named, and the current row left alone until a person answers. A contract with an empty second page comes back held, and the note names page 2. The parties and the effective date can still be cited. The file is not accepted.

The demo is smaller on purpose. It proves the state changes that decide whether a number becomes truth. Production extends that same machine with durable doors, a real login, a scan, OCR, and chunks that keep the page on every field. I would rather show a hold I can explain than a pile of integrations I cannot defend.

REFERENCES

n8n, Webhook node. docs.n8n.io. The intake, confirm, book, and thread triggers.

n8n, Form trigger and Manual trigger. [Form](#), [Manual](#). The distinction in A1.

n8n, Queue mode. docs.n8n.io/hosting/scaling/queue-mode. Redis and a worker. This build runs that way, on port 5680, separate from any other n8n on the machine.

n8n, HTTP Request node, retry on fail. docs.n8n.io. Four tries, three seconds apart, on the model call.

Google, Gemini API pricing, checked 22 September 2026. ai.google.dev/gemini-api/docs/pricing. Flash 0.30 / 2.50 per million tokens. Flash-Lite 0.10 / 0.40. Output price includes thinking tokens.

OpenRouter, Gemini 2.5 Flash, same rates on that date. openrouter.ai/google/gemini-2.5-flash. The route the workflow calls.

Token counts in the cost table are estimates, not a measured bill from this project.